

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Programação Funcional em Python

Do paradigma à prática — estudo de caso de um sistema real em produção

Juliano Freitas

Faculdade Nova Roma

O caminho de hoje

1 O que é e por que importa

3 Funções como valores

5 functools

7 Avaliação preguiçosa

9 Estudo de caso: o projeto

2 Os pilares

4 Ferramental do dia a dia

6 Closures e decoradores

8 Recursão

10 PF × Imperativo e limites

OBJETIVOS

Ao final, você será capaz de:



Identificar

Reconhecer código funcional e seus conceitos em Python real.



Escrever

Aplicar funções puras, HOFs, closures e decoradores com propósito.



Justificar

Explicar POR QUE o estilo funcional ajuda — e onde ele não compensa.

A aula usa um sistema real — o projeto Ponte Italia — como fio condutor: cada conceito aparece em código que roda.

01

O que é e por que importa

Paradigmas, origem e a ideia central da programação funcional

Duas formas de dizer a mesma coisa

Imperativo — COMO fazer

```
total = 0
for n in numeros:
    total += n
```

Passo a passo, mudando estado.

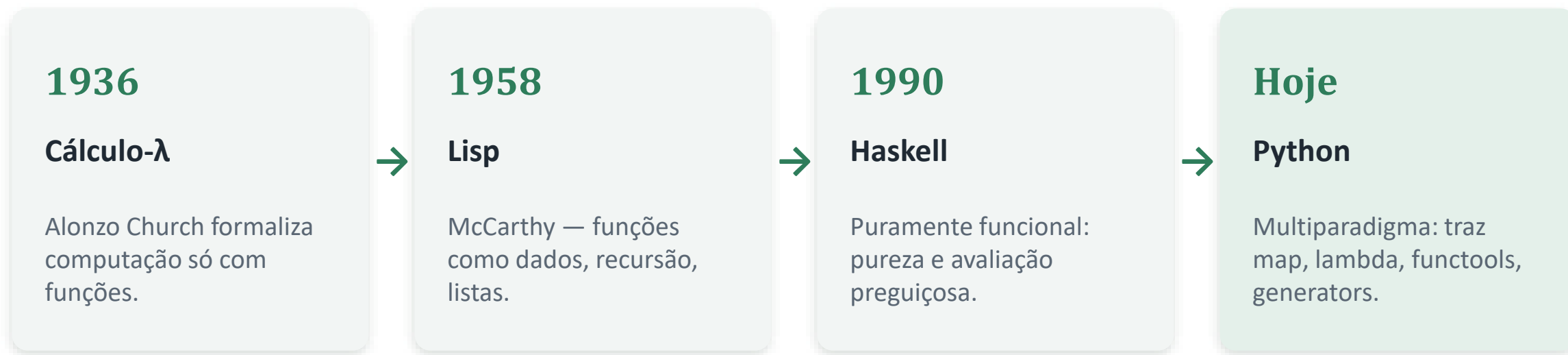
Funcional — O QUE se quer

```
total = sum(numeros)
```

Descreve o resultado; sem estado intermediário exposto.

A programação funcional é um estilo declarativo: compomos funções em vez de encadear comandos que alteram variáveis.

De uma teoria dos anos 1930 ao Python de hoje



Três compromissos

1

Funções no centro

O programa é a avaliação e composição de funções, não uma lista de comandos.

2

Evitar estado mutável

Preferir dados imutáveis: em vez de alterar, produzir um novo valor.

3

Sem efeitos colaterais

A função conversa com o mundo pelo retorno — não por variáveis externas.

Python não é Haskell — e tudo bem

Python é multiparadigma. A boa engenharia não é ser 100% funcional, e sim usar o estilo funcional ONDE ele paga.

Funcional onde calcula

- Regras de negócio
- Transformação de dados
- Cálculos testáveis

Imperativo nas bordas

- Ler/gravar banco
- Requisições HTTP
- Arquivos e relógio

02

Os pilares

Pureza, efeitos colaterais, imutabilidade e transparência referencial

Função pura

- Mesma entrada → mesma saída, sempre.
- Não lê nem altera nada fora dela (sem efeitos).
- Depende só dos argumentos que recebe.

```
# pura
def dobro(x):
    return x * 2

# impura: depende de estado externo
total = 0
def acumula(x):
    global total
    total += x          # efeito
    colateral
    return total
```

O que é um efeito colateral



Mutar estado externo

variável global, atributo, lista compartilhada



I/O

imprimir, ler banco, requisição de rede, arquivo



Depender do ambiente

hora atual, número aleatório, variável de ambiente

Efeitos não são proibidos — são empurrados para as BORDAS do sistema. O núcleo que calcula fica puro, previsível e testável.

Não altero — crio um novo

- tuple, frozenset, str: imutáveis de fábrica.
- `@dataclass(frozen=True)`: objetos que não mudam.
- Sem mutação = sem bug de estado compartilhado.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class DocumentoResumo:
    categoria: Categoria
    status: Status

d = DocumentoResumo(cat, ok)
d.status = venc    # ERRO: congelado
```

Trocar a chamada pelo seu valor

```
preco(3)  -> 15.0          # sempre  
  
# então estas duas linhas são iguais:  
total = preco(3) + preco(3)  
p = preco(3)  
total = p + p
```

Por que importa

- Dá para raciocinar sobre o código como em álgebra.
- Habilita cache seguro (memoização).
- Permite paralelizar sem travas.

É a propriedade que só funções puras têm — e a base dos slides de `lru_cache` mais à frente.

calcular_status — o relógio entra pela porta

```
def calcular_status(
    validade: date | None,
    hoje: date,                # injetado!
    janela: int = 30,
) -> StatusDocumento:
    if validade is None:
        return Status.OK
    if validade < hoje:
        return Status.VENCIDO
    if validade <= hoje + dias(janela):
        return Status.VENCENDO
    return Status.OK
```

Por que pura

- hoje é parâmetro, não datetime.now().
- O status não é gravado no banco: é derivado na leitura.
- Testável sem relógio, sem mock, sem banco.

Três ganhos que caem no seu colo

01

Testabilidade

Um teste = três linhas. Sem fixture, sem mock, sem banco de pé.

02

Concorrência segura

Sem estado compartilhado, dois pedidos simultâneos não colidem.

03

Raciocínio local

A regra inteira cabe na tela; nada acontece 'à distância'.

03

Funções como valores

Primeira classe, funções de alta ordem e composição

Funções são dados

- Guardar em variável, lista ou dicionário.
- Passar como argumento.
- Retornar de outra função.

```
def dobro(x): return x * 2

f = dobro          # sem parênteses!
f(10)              # 20

ops = {"x2": dobro, "neg": lambda x: -
x}
ops["x2"](5)       # 10
```

Funções de alta ordem (HOF)

HOF = uma função que recebe outra função como argumento e/ou devolve uma função.

```
# map, filter, sorted já são HOFs da linguagem:  
list(map(str.upper, nomes))  
list(filter(lambda n: n.ativo, usuarios))  
sorted(cursos, key=lambda c: c.prazo)    # recebe função
```

O registry de parsers

```
# scraper/sources/__init__.py
PARSERS = {
    "universitaly": parser_universitaly,
}

# a função é escolhida pelo NOME, como valor:
parser = PARSERS[fonte]
universidades = parser(html)
```

O ganho

- Zero if fonte == '...' espalhado.
- Fonte nova = uma linha no dicionário.
- O código de coleta nunca muda.

agrupar_por_categoria recebe a chave

```
def agrupar_por_categoria(itens, chave):  
    return {  
        cat: tuple(i for i in itens  
                    if chave(i) == cat)  
        for cat in Categoria  
    }  
  
# a MESMA função serve documentos e requisitos:  
agrupar_por_categoria(docs, lambda d:  
d.categoria)
```

Composição

Funções puras se encaixam como peças:

```
calcular_status  
    ↓  
calcular_gap
```

04

Ferramental do dia a dia

lambda, map, filter e comprehensions

Função anônima, curtinha

```
quadrado = lambda x: x ** 2
quadrado(5)      # 25

# uso típico: como argumento inline
sorted(pares, key=lambda p: p[1])
```

PEP 8

- lambda só inline, como argumento.
- Precisa de nome? Use def.
- Uma expressão — sem if/for de bloco.

Transformar e selecionar sem loop

map — transforma cada item

```
list(map(str.strip, linhas))  
  
# equivale a:  
[l.strip() for l in linhas]
```

filter — mantém quem passa

```
list(filter(eh_valida, urls))  
  
# equivale a:  
[u for u in urls if eh_valida(u)]
```

Em Python, a comprehension costuma ser a forma mais legível — veja a seguir.

ordenar_por_prazo

```
def ordenar_por_prazo(cursos, prazo_de):  
    return tuple(sorted(  
        cursos,  
        key=lambda c: (prazo_de(c) is None,  
                       prazo_de(c) or date.max),  
    ))
```

A regra na chave

- Quem tem prazo vem antes.
- Prazo mais próximo primeiro.
- prazo_de é injetado: serve a vários tipos.

O jeito pythônico de transformar dados

```
quadrados = [x*x for x in range(10)]           # list
por_id     = {u.id: u for u in usuarios}        # dict
dominios   = {e.split("@")[1] for e in emails}  # set

ativos = [u.nome for u in usuarios if u.ativo]  # com filtro
```

Lê-se quase como português: 'o nome de cada usuário, para cada usuário, se ativo'.

Achatando requisitos por categoria

```
avaliacoes = tuple(  
    _avaliar(req, por_cat.get(cat, ()))  
    for cat, grupo in requisitos_por_categoria(reqs).items()  
    for req in grupo  
)
```

Dois for aninhados e uma chamada pura por item — nenhuma lista temporária mutada, nenhum índice manual.

05

functools

reduce, partial e lru_cache — a caixa de ferramentas funcional

Dobrar uma sequência num único valor

```
from functools import reduce

total = reduce(
    lambda acc, n: acc + n.peso,
    noticias, 0,
)
```

Como pensar

- Acumulador + próximo item → novo acumulador.
- sum, max, any são reduces especializados.
- Comece pelo built-in; reduce para o resto.

Fixar argumentos e nascer uma função nova

```
from functools import partial

def conectar(host, porta, tls): ...

conectar_local = partial(conectar, "127.0.0.1", 5432)
conectar_local(tls=False)    # host e porta já fixos
```

Especializar sem herança, sem classe, sem repetir código: a nova função é a antiga com parte dos argumentos pré-preenchidos.

Um parser por fonte, sem duplicar código

```
from functools import partial

parser_universitaly = partial(
    parse_universidades,          # parser genérico
    fonte="universitaly",
    seletores=SELETORES_UNIVERSITALY,
)
```

O parser genérico sabe transformar HTML em tuplas imutáveis; cada fonte é só uma aplicação parcial com seus seletores. Fonte nova não copia código — só pré-preenche argumentos.

Memoização em uma linha

```
from functools import lru_cache

@lru_cache(maxsize=256)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)
```

A pegadinha

Só é CORRETO memoizar função pura. Cachear algo que lê o banco devolve dado velho — vira bug de dados obsoletos.

requisitos_por_categoria — cache que é seguro por desenho

```
@lru_cache(maxsize=256)
def requisitos_por_categoria(reqs):
    return MappingProxyType(          # saída imutável
        agrupar_por_categoria(
            reqs, lambda r: r.categoria))
```

Seguro porque...

- Entrada imutável e hashável (tupla).
- Função determinística, sem efeitos.
- Saída imutável (MappingProxyType).

O mesmo curso é comparado por muitos alunos: indexa uma vez, não uma vez por pedido.

06

Closures e decoradores

Funções que capturam ambiente e funções que envolvem funções

Uma função que lembra do ambiente

```
def multiplicador(fator):  
    def aplicar(x):  
        return x * fator    # captura 'fator'  
    return aplicar  
  
triplo = multiplicador(3)  
triplo(10)    # 30
```

A ideia

A função interna 'fecha sobre' as variáveis de fora e as carrega consigo. É uma fábrica de funções configuradas.

Uma função que envolve outra função

```
def log(funcao):  
    @wraps(funcao)                # preserva nome/doc  
    def wrapper(*args, **kwargs):  
        print("chamando", funcao.__name__)  
        return funcao(*args, **kwargs)  
    return wrapper  
  
@log  
def somar(a, b): return a + b
```

@log é açúcar para `somar = log(somar)`. O decorador adiciona comportamento transversal (log, tempo, auth) sem tocar na função original.

@cronometrar mede sem poluir a função

```
def cronometrar(funcao):  
    @wraps(funcao)  
    def wrapper(*args, **kwargs):  
        inicio = perf_counter()  
        try:  
            return funcao(*args, **kwargs)  
        finally:  
            # mede até se  
            dur = (perf_counter()-inicio)*1000  
            logger.info("%.1f ms", dur)  
    return wrapper
```

Detalhes

- @wraps mantém introspecção e docs.
- finally garante a medição mesmo com exceção.
- Reusável em qualquer função.

retry_backoff — closure parametrizada

```
def retry_backoff(tentativas=3, base=0.5,
                  fator=2.0, dormir=time.sleep):
    def decorador(funcao):
        @wraps(funcao)
        def wrapper(*args, **kwargs):
            for i in range(tentativas):
                try:
                    return funcao(*args, **kwargs)
                except Exception:
                    if i == tentativas-1: raise
                    dormir(base * fator**i)
            return wrapper
        return decorador
```

Closure + teste

- Cada uso captura sua própria config.
- dormir é injetado: testa o backoff sem dormir de verdade.

07

Avaliação preguiçosa

Generators: produzir sob demanda, gastar memória mínima

yield: produz um de cada vez

```
def contador(n):  
    i = 0  
    while i < n:  
        yield i          # pausa e devolve  
        i += 1  
  
soma = sum(x*x for x in range(10**6))  
# gera sob demanda, sem criar a lista toda
```

Por quê

- Memória $O(1)$: não materializa tudo.
- Trabalha com fluxos infinitos.
- `()` = generator; `[]` = lista.

Feed de notícias item a item

- Exemplo previsto — Sprint 4 (Radar)

```
def itens_do_feed(paginas):  
    for pagina in paginas:  
        for noticia in pagina:  
            yield normalizar(noticia)    # preguiçoso  
  
primeiras = islice(itens_do_feed(paginas), 20)
```

A paginação do feed lê e normaliza só o necessário para a página pedida — nada de carregar todas as notícias na memória.

08

Recursão

Um problema definido em termos de si mesmo

Achatando categorias aninhadas do FAQ

- Exemplo previsto — Sprint 6 (FAQ)

```
def achatatar(categoria):  
    perguntas = list(categoria.perguntas)  
    for sub in categoria.subcategorias: # aninhadas  
        perguntas += achatatar(sub)    # passo  
    return perguntas    # base: sem subcategorias
```

Em Python

- Caso base + passo recursivo.
- Limite ~1000 (sys.setrecursionlimit).
- Sem otimização de cauda (TCO).

09

Estudo de caso: Ponte Italia

Onde todos os conceitos se encontram em código que roda

Cálculo puro no centro, efeitos nas bordas

scraper/

rede · HTML

NÚCLEO PURO

services/ – calcular_gap,
calcular_status, comparativo

routers/

HTTP · banco

db/

MySQL

As bordas fazem I/O e entregam dados imutáveis ao núcleo; o núcleo só calcula e devolve. Testo o núcleo sem subir nada.

calcular_gap — o cálculo mais importante do sistema

```
def calcular_gap(requisitos, documentos) -> Gap:
    por_cat = agrupar_por_categoria(
        documentos, lambda d: d.categoria)
    aval = tuple(
        _avaliar(req, por_cat.get(cat, ()))
        for cat, grupo in por_categoria(requisitos)
        for req in grupo)
    tri = lambda s: tuple(i for st,i in aval
                          if st == s)
    return Gap(tri("atendido"),
               tri("faltando"), tri("vencendo"))
```

(trecho condensado)

100% pura

- Tuplas congeladas entram, Gap congelado sai.
- Não conhece banco, HTTP nem relógio.
- 14 testes cobrem todos os cruzamentos.

Cada conceito vive em código real

Função pura	<code>calcular_gap, calcular_status</code>
Imutabilidade	<code>schemas frozen, dataclasses</code>
Primeira classe / HOF	<code>registry PARSERS, agrupar_por_categoria</code>
lambda	<code>ordenar_por_prazo</code>
partial	<code>parser_universitaly</code>
lru_cache	<code>requisitos_por_categoria</code>
Decorador / closure	<code>@cronometrar, retry_backoff, @exigir_auth</code>
Generators (previsto)	<code>feed lazy · Sprint 4</code>
Recursão (previsto)	<code>achatar FAQ · Sprint 6</code>

10

PF × Imperativo e limites

O contraste lado a lado e onde a PF não compensa

COMPARAÇÃO

O mesmo cálculo, dois mundos

Funcional — puro

```
def calcular_gap(reqs, docs):  
    ...                # só cálculo  
    return Gap(...)  
  
# testa sem banco, sem mock
```

Imperativo — acoplado

```
reqs = sessao.execute(SQL)  
for req in reqs:  
    docs = sessao.execute(SQL) # N+1  
    if doc.status == "ok": # status  
        velho  
        atendidos.append(req)
```

A versão imperativa mistura buscar + derivar + particionar, depende de status defasado no banco e exige um MySQL de pé para cada teste.

LIMITES

Quando a PF NÃO compensa em Python

Cópias em excesso

Recriar grandes estruturas a cada passo pode custar memória. Às vezes mutar local é ok.

Recursão profunda

Sem TCO e com limite ~1000, laços são mais seguros que recursão longa.

Legibilidade

map/filter/reduce encadeados podem ficar ilegíveis — comprehension costuma vencer.

I/O é inevitável

Todo sistema real tem efeitos. O objetivo é isolá-los, não fingir que não existem.

Como aplicar isso sem exagero

- Separe o cálculo do I/O: núcleo puro, efeitos nas bordas.
- Dados que atravessam funções: prefira imutáveis (tuple, frozen).
- Funções pequenas e nomeadas; lambda só inline.
- Só memoize (lru_cache) função pura.
- Deixe ruff e mypy vigiarem tipos e estilo.

Três frases para guardar

1 Função pura = mesma entrada, mesma saída, sem efeitos. É a base de tudo.

2 Funções são valores: passe-as, retorne-as, guarde-as — e componha.

3 Isole os efeitos nas bordas; mantenha o núcleo puro, testável e memoizável.

Obrigado!

Perguntas são bem-vindas.

Projeto Ponte Italia ·

github.com/julianosfreitas/scrapping_italy

Fontes e recursos

- Slides de autoria própria, a partir do código do projeto Ponte Italia (autoria do apresentador).
- Documentação oficial do Python — `functools`, `itertools`, `dataclasses` (docs.python.org).
- PEP 8 — Guia de Estilo para Python.

[n] Reservado para slides gerados por IA: indicar a IA usada e o prompt exato. *Ex.: [1] ChatGPT — "gerar diagrama de arquitetura em camadas".*